

# HGRL: Human-Driving-Data Guided Reinforcement Learning for Autonomous Driving

Hejian Zhuang<sup>ID</sup>, Hongqing Chu<sup>ID</sup>, Yafei Wang<sup>ID</sup>, *Member, IEEE*, Bingzhao Gao<sup>ID</sup>, *Member, IEEE*,  
and Hong Chen<sup>ID</sup>, *Fellow, IEEE*

**Abstract**—Reinforcement learning (RL) shows promise for autonomous driving decision-making. However, designing appropriate reward functions to guide RL agents towards complex optimization objectives is challenging. This article proposes a framework that learns the reward function from human driving data to guide RL agent's learning. The proposed framework consists of three components: trajectory sample, offline preference learning, and RL. Firstly, feasible trajectories are generated by sampling end targets from a reachable state space. Subsequently, a novel offline preference learning framework is utilized to train a transformer network by comparing generated feasible trajectories with human driving trajectories. The transformer network is used to model the human driving decision-making process, thereby obtaining a reward function. Finally, to obtain the final driving decision network, the derived reward function is incorporated into a RL framework. To validate the proposed method, a highway simulator is established where the surrounding vehicle trajectories are derived from real-world driving scenarios. Compared with baseline algorithms, the proposed method achieves the best performances in terms of decision safety and human-likeness. Additionally, the learned policy network performs well in driving decision-making tasks with longer total decision steps. Experimental results demonstrate that the proposed method can obviate the requirement for manual design of sophisticated reward functions in RL-based autonomous driving decision-making systems.

**Index Terms**—Autonomous driving, reinforcement learning (RL), reward function, preference learning.

## I. INTRODUCTION

WITH the rapid advancement of artificial intelligence, autonomous driving is regarded as one of the most promising technologies of the automotive industry [1], [2], [3]. As an integral component of autonomous driving, the decision-making system represents a focal and challenging aspect in current research. The performance of the decision-making system

is crucial to the overall driving performance of autonomous vehicles and is often referred to as the brain of autonomous driving [4], [5].

Reinforcement learning (RL) is a widely utilized method in autonomous vehicle (AV) decision-making systems [6], due to its capacity to learn complex policies in high-dimensional environments [7], [8], [9]. Through interactions with the driving environment, the RL agent can learn driving policies without explicitly modeling the complex environment [10]. However, there are still challenges that need to be addressed. One major challenge is how to design reward functions of RL that can accurately guide the learning direction of RL agents towards intricate optimization objectives in the complex task of autonomous driving decision-making [11], [12], [13]. The reward function is frequently regarded as the task's most concise description [14], and it guides the learning direction of RL. If the reward function is poorly designed, RL agents may receive a large reward return employing an unexpected policy [15], [16]. Moreover, the environment faced by AVs is complex, involving interactions between vehicles and pedestrians. This complexity further exacerbates the challenges of manually designing appropriate reward functions.

A popular approach to avoid this challenge is to leverage human driving data or human feedback data to assist RL algorithms in learning [11]. Human driving data or human feedback data contains driver decision-making information. Mining and utilizing this information can assist RL agents and lead to the learning of driving policies. RL with demonstrations integrates expert demonstration data into the RL framework, yet it still necessitates reward function design [15]. Preference learning [17], on the other hand, utilize human feedback data introduced during the RL training process to train the reward function, addressing the challenge of manually designing reward functions. However, obtaining human feedback data during the RL training process can be costly [18]. In general, human driving data is easier to obtain compared to human feedback data in the field of AV. Inverse reinforcement learning (IRL) can leverage human driving data to simplify the design of the reward function and even learn driving policies without the need for manual reward function design [19]. However, both preference learning and IRL methods involve joint learning of the reward function and policy network [17], [19], which can be inefficient in the RL training process.

Motivated by the above challenge, this article proposes a human-driving-data guided RL framework for decision-making

Manuscript received 26 March 2024; revised 17 May 2024; accepted 23 May 2024. Date of publication 29 May 2024; date of current version 11 July 2025. This work was supported in part by the National Nature Science Foundation of China under Grant 62273256 and Grant 62373289, and in part by the Fundamental Research Funds for the Central Universities. (Corresponding author: Hongqing Chu.)

Hejian Zhuang, Hongqing Chu, and Bingzhao Gao are with the School of Automotive Studies, Tongji University, Shanghai 201804, China (e-mail: zhjph\_2@163.com; chuhongqing@tongji.edu.cn; gaobz@tongji.edu.cn).

Yafei Wang is with the School of Mechanical Engineering, Shanghai Jiaotong University, Shanghai 200240, China (e-mail: wyfjlu@sjtu.edu.cn).

Hong Chen is with the College of Electronics and Information Engineering, Tongji University, Shanghai 201804, China (e-mail: chen hong2019@tongji.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TIV.2024.3406679>.

Digital Object Identifier 10.1109/TIV.2024.3406679

in AVs. This framework uses human driving data to offline learn the reward function, which subsequently guides the learning of driving policies during the training phase of RL. First of all, to provide an training and evaluation environment for human-like decision-making systems in AVs, we create a simulator incorporating real-world interactive information in a highway scenario. Then, to address the inefficiency caused by joint learning of the reward function and policy network, this article propose an offline preference learning framework to learn the reward function prior to the RL training phase. The framework models the human driving decision-making process by comparing generated feasible driving trajectories with human trajectories, ultimately resulting in the generation of a reward function. Finally, the derived reward function is incorporated into a RL framework to obtain the final policy network. The main contributions of this article are listed as follows:

- 1) This article proposes a human driving data guided RL framework, which utilizes human driving data to train the reward function, subsequently applying it to guide the training of RL agents. The proposed framework obviates the requirement for manual design of sophisticated reward function in RL-based autonomous driving decision-making system and achieves better performance than baseline algorithms.
- 2) This study proposes an offline preference learning framework for training the reward function, which, unlike conventional preference learning approaches, allows for the acquisition of the reward function prior to the RL training phase through training on an offline dataset. The efficiency of the RL training process can be enhanced.
- 3) This article constructs an offline dataset for preference learning by comparing human driving trajectories with generated feasible trajectories. This allows the utilization of exiting human driving trajectory datasets for training reward functions within preference learning frameworks, reducing the cost of providing human feedback that is needed in conventional preference learning approaches.

The remainder of this article is organized as follows. Related work in recent years is introduced in Section II. The problem formulation and background are described in Section III. Section IV presents the proposed HGRL algorithm. Then, experiments and results are presented in Section V. Finally, conclusions and future works are summarized in Section VI.

## II. RELATED WORK

Since the decision-related information inherent in the human driving data or human feedback data can greatly assist RL based algorithms in achieving better autonomous driving decision-making, several algorithms have been developed to incorporate this information into the RL framework. Among them, RL with expert demonstrations, IRL, and preference learning are considered three promising approaches.

### A. RL With Expert Demonstrations

Expert demonstration data, which contains human decision-making processes, can improve the performance of RL based methods. Existing methods usually utilize expert demonstrations for pre-training the policy through supervised learning or add expert demonstration data into the experience replay buffer [15]. Liang et al. [20], built a constrained action space guided by imitating human demonstrations to improve the RL agent exploration efficiency. Another study [21] added expert demonstration data into the replay buffer and utilized a prioritized replay mechanism to enhance the performance of RL. Liu et al. [22] combined these two approaches by utilizing expert demonstration data and the soft actor-critic RL framework to achieve longitudinal control for autonomous vehicles. Some studies also incorporate human demonstrative behaviors in real-time into the training process of RL, which is a form of human-in-the-loop RL. Wu et al. [23] introduced human guidance actions into both real-time manipulation and the offline learning process of RL to achieve autonomous driving. However, the primary intention behind these algorithms is often to accelerate the exploration efficiency and they still necessitate reward function design.

### B. Inverse Reinforcement Learning

IRL attempts to incorporate human data into the RL framework to learn decision policies without the need for meticulous reward function design. Maximum entropy IRL and Generative Adversarial Imitation Learning (GAIL) are two types of IRL that are frequently employed in the field of autonomous driving [24]. In some studies [25], [26], the coefficients of a collection of handcrafted features are adjusted using maximum entropy IRL to provide a reward function for evaluating generated driving trajectories. However, handcrafted feature combinations can not capture human driving decision information well [15]. GAIL [27] learns the policy through the adversarial learning between a discriminator and a RL agent. Wang et al. [28] employed this method to allow AV decision-making in Highway environment. However, due to adversarial learning, GAIL is difficult to train and requires heavy computation [24]. To avoid these limitations, the proposed approach decouples the training of the reward function from the RL training. The reward function is trained prior to RL training using an offline dataset. And a neural network is employed as the carrier for the reward function to better capture human driving decision-making information.

### C. Preference Learning

The preferences of human decision-making are contained in the data from human feedback. The reward function serves as the most concise description of the task [14] and also reflects the preferences regarding the decision-making process. Some studies [11], [12], [17] utilized this characteristic to learn the reward function through receiving human-in-the-loop feedback during the RL training phase. And their works show that the RL agent can avoid unexpected exploitation and learn novel behaviors by employing the reward function learned through preference learning. And preference learning is one of the main

technologies behind ChatGPT 3 [29]. Due to the success of ChatGPT, this technology is currently receiving widespread attention [30]. However, providing real-time human feedback data during the RL training phase is costly. Within the domain of AV, human driving data is more accessible, compared to human feedback data. And the joint learning of the reward function and policy network through human-in-the-loop feedback leads to slow data collection and training efficiency [16]. Inspired by the success of preference learning and to avoid the aforementioned limitations, a novel offline preference learning framework is proposed to learn the reward function. We train the reward function prior to RL training by preferring real human trajectory with generated trajectories under the proposed offline preference learning framework.

### III. PROBLEM FORMULATION AND BACKGROUND

#### A. Problem Formulation

The focus of this article is to address the refined driving decision problem, specifically determining a policy based on input information about the ego vehicle, surrounding vehicles, and the road. The objective is to output appropriate continuous control quantities (steering angle and acceleration) to achieve human-like and safe autonomous driving. The policy can be modified as follows:

$$(a_t, \delta_{ft}) = \pi_\theta(s_t), \quad (1)$$

where  $a_t, \delta_{ft}$  are the acceleration and front steering angle control outputs for the ego vehicle at timestep  $t$ , and  $s_t$  is the state at timestep  $t$ , which includes the information about ego vehicle, surrounding vehicles and the current road. The kinematics of the ego vehicle are modeled based on the bicycle model:

$$\begin{bmatrix} \dot{x}_t \\ \dot{y}_t \\ \dot{\beta}_t \end{bmatrix} = v_t \times \begin{bmatrix} \cos \beta_t \\ \sin \beta_t \\ \frac{\tan \delta_{ft}}{L} \end{bmatrix}, \quad \dot{v}_t = a_t, \quad (2)$$

where  $x_t$  is the longitudinal position of ego vehicle at timestep  $t$ ,  $y_t$  is the lateral position of ego vehicle at timestep  $t$ ,  $\beta_t$  is the heading angle of ego vehicle at timestep  $t$ , and  $L$  is the wheelbase of ego vehicle. The objective of the policy is to generate control outputs  $(a, \delta_f)$  that, when applied to control ego vehicle (modeled based on the bicycle model), produce a trajectory  $\sigma^1$  that closely matches the trajectory  $\sigma^0$  demonstrated by the human driver under the same driving scenario, i.e.,

$$\pi^* = \arg \min_{\theta} (\text{diff}(\sigma^0, \sigma^1)). \quad (3)$$

#### B. Background

1) *Preference Learning*: Preference learning utilizes neural networks to excavate decision preferences from human preference data, thereby learning the reward function. The learning process of preference learning is introduced as follows. A segment  $\sigma$  is a sequence of states and actions  $\{(s_1, a_1), \dots, (s_n, a_n)\}$ . When presented with two segments

$(\sigma^0, \sigma^1)$ , humans exhibit preferences between them. These preferences can be captured by assigning preference labels to the segment pairs, i.e.,  $y \in \{(0, 1), (1, 0), (0.5, 0.5)\}$ . The preference label  $y = (0, 1)$  indicates that humans prefer  $\sigma^1$  ( $\sigma^0 \prec \sigma^1$ ),  $y = (1, 0)$  indicates that humans prefer  $\sigma^0$  ( $\sigma^0 \succ \sigma^1$ ), and  $y = (0.5, 0.5)$  indicates no differential preference between the two segments ( $\sigma^0 \sim \sigma^1$ ). A dataset  $\mathcal{D}$  records the preferences for each pair of segments as a triple  $(\sigma^0, \sigma^1, y)$ . Following the Bradley-Terry model [31], a preference predictor can be modeled using the reward function  $r_\psi$  as follows:

$$P_\psi[\sigma^0 \prec \sigma^1] = \frac{\exp(\sum_t r_\psi(s_t^1, a_t^1))}{\sum_{i \in \{0,1\}} \exp(\sum_t r_\psi(s_t^i, a_t^i))}. \quad (4)$$

As shown in (4), the probability of a segment being preferred exhibits an exponential relationship with the cumulative reward of the segment. Based on this, the reward function, modeled as a neural network with parameters  $\psi$ , can be optimized by maximizing the probability of the preferred segment through minimizing the following loss function:

$$\begin{aligned} \mathcal{L}(\psi) = & - \mathbb{E}_{(\sigma^0, \sigma^1, y) \sim \mathcal{D}} [y(0) \log P_\psi[\sigma^0 \succ \sigma^1] \\ & + y(1) \log P_\psi[\sigma^0 \prec \sigma^1]]. \end{aligned} \quad (5)$$

2) *Reinforcement Learning*: RL aims to find the optimal control policy by maximizing cumulative rewards in dynamic environments. And it can be defined by Markov decision processes (MDPs). An MDP can be represented as a tuple  $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma \rangle$  where  $\mathcal{S}$  is the state space with  $s \in \mathcal{S}$ ,  $\mathcal{A}$  is the action space with  $a \in \mathcal{A}$ ,  $\mathcal{P}$  is the state transition probability matrix that provides the probability of the system transition between each pair of states and actions  $\mathcal{P}(s' | s, a)$ ,  $r$  is the reward function  $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ , and  $\gamma$  is a discount factor  $\gamma \in (0, 1]$ . The goal of the RL is to find the optimal policy  $\pi^*$  that maximizes cumulative rewards and can be defined as follows:

$$\pi^* = \arg \max \mathbb{E} \left[ \sum_{t=0}^T \gamma^t r(s_t, \pi(s_t)) \right]. \quad (6)$$

With advances in deep neural networks, deep RL has achieved remarkable successes in complex decision-making tasks [7].

### IV. HUMAN-DRIVING-DATA GUIDED REINFORCEMENT LEARNING

#### A. Framework

Fig. 1 shows the conceptual framework of our proposed method HGRL. Our method consists of three key steps: 1) a method named trajectory sample to create an offline dataset for reward function learning; 2) an offline preference learning framework to learn the reward function for RL; and 3) RL to obtain the final policy network with the guidance of the reward function learned from human driving data. Firstly, polynomial trajectory generation is utilized to sample reachable states and generate potential driving trajectories. An offline dataset is established by labeling human trajectories and human-like trajectories with preference labels, while labeling the remaining trajectories with non-preference labels.



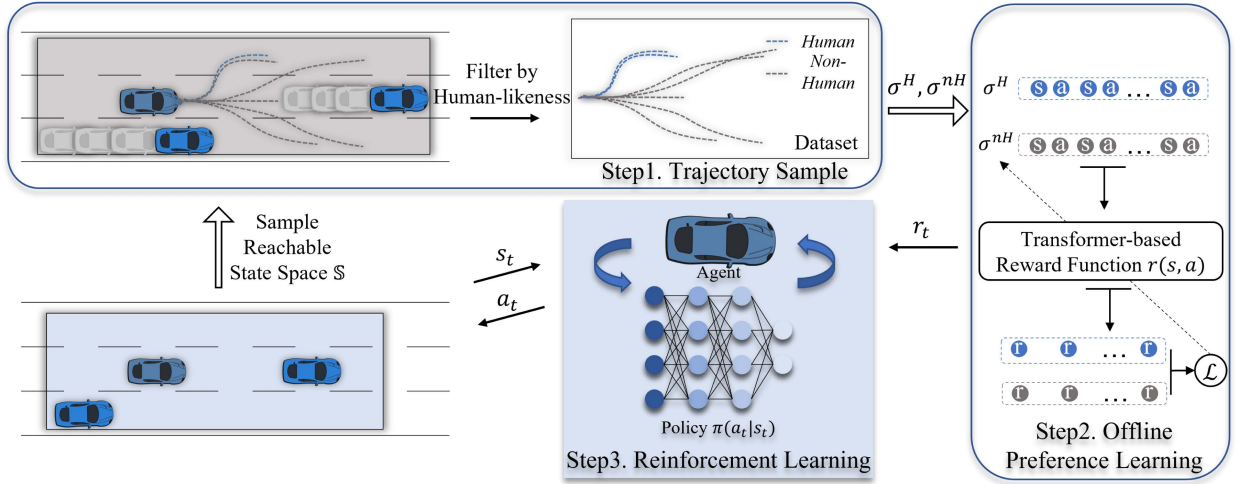


Fig. 1. Conceptual framework of HGRL.

An offline preference learning framework is subsequently employed to train a transformer network on this dataset, yielding a reward function. The combination of the offline preference learning framework, which excels in sequence comparison, and transformer networks, which excel in sequence modeling, enables effective modeling of the human driving decision-making process and extraction of an accurate reward function. This reward function is then integrated into an RL framework to guide policy learning. By decoupling the reward function learning from the RL training, our method is distinct from conventional preference learning and some IRL algorithms like GAIL. It is beneficial for enhancing the efficiency of the training process. Finally, a human-like policy network is obtained.

### B. Trajectory Sample

In conventional preference learning, the segment dataset with preference labels is acquired through agent-environment interactions during the RL training process, with labels provided by human feedback. This online approach leads to slow training efficiency and high cost. Moreover, the limited segments available in the early training phase may hinder properly learning the reward model, destabilizing RL training. To address these limitations and motivated by the fact that human driving trajectories are readily accessible in the field of autonomous driving, we propose a novel method to build the dataset that is then utilized in an offline preference learning framework to train a reward function. We first sample target states  $\{s_\tau^1, s_\tau^2, \dots\}$  from the reachable state space  $\mathbb{S}$  base on the endpoint  $s_\tau^0$  of real human trajectory over a time horizon  $\tau$ . These target states are then used to generate trajectories  $\{\sigma^1, \sigma^2, \dots\}$  by fitting polynomial curves, as visualized in Fig. 2. Then, a controller tracks the generated trajectories and real human driving trajectories to obtain segments of state-action pairs. Finally, segments generated based on human trajectories and those similar to human trajectories are labeled as preference, while the remaining segments

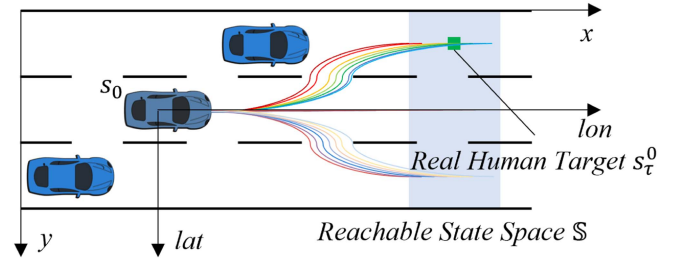


Fig. 2. The process of trajectory generation.

are labeled as non-preference. By decoupling dataset making from the RL phase, our approach enables effective and efficiency reward function offline learning before policy training.

Taking into account the road structure (available lane center-lines), semantics (lane-changing, car-following, and different speed profiles), traffic rules, and dynamic feasibility, the potential trajectories are generated by using polynomial curves with given end targets [32], [33]. And the given end targets are sampled from the reachable state space. We first represent the trajectory in the Frenet Frame of the reference path (lane centerline, for example) [34] and then translate it back to the Cartesian coordinate, which allows us to adapt to intricate road structures. The trajectory in the Frenet Frame can be represented as follows:

$$\begin{cases} lon(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 \\ lat(t) = b_0 + b_1t + b_2t^2 + b_3t^3 + b_4t^4 + b_5t^5 \end{cases}, \quad (7)$$

where  $lon(t), lat(t)$  represent the longitudinal and lateral positions of the vehicle at time  $t$ , respectively, and  $\{a_0, a_1, a_2, a_3, a_4\}, \{b_0, b_1, b_2, b_3, b_4, b_5\}$  are the coefficients. The coefficients can be calculated using the start state  $s_0 = (0, v_{lon0}, a_{lon0}, 0, v_{lat0}, a_{lat0})$  and given end target state  $s_e =$

$(v_{lon_e}, a_{lon_e}, lat_e, v_{late}, a_{late})$ , as shown in (8):

$$\begin{cases} lon(t=0) = 0 \\ \dot{lon}(t=0) = v_{lon0} \\ \ddot{lon}(t=0) = a_{lon0} \\ lon(t=\tau) = v_{lon_e} \\ \ddot{lon}(t=\tau) = a_{lon_e} \end{cases}, \begin{cases} lat(t=0) = 0 \\ \dot{lat}(t=0) = v_{lat0} \\ \ddot{lat}(t=0) = a_{lat0} \\ lat(t=\tau) = lat_e \\ \dot{lat}(t=\tau) = v_{late} \\ \ddot{lat}(t=\tau) = a_{late} \end{cases}. \quad (8)$$

The given end target state  $s_e$  is sampled from the reachable state space  $\mathbb{S}$ , which can be defined as follows:

$$\mathbb{S} = \{(v_{lon}, a_{lon}, lat, v_{lat}, a_{lat}) \mid v_{lon} \in \mathbb{V}_{lon}, a_{lon} \in \mathbb{A}_{lon}, lat \in \mathbb{L}, v_{lat} \in \mathbb{V}_{lat}, a_{lat} \in \mathbb{A}_{lat}\}, \quad (9)$$

where  $\mathbb{V}_{lon}$  denotes the set of possible longitudinal velocities, which is defined as:

$$\mathbb{V}_{lon} = \text{linspace}([\max(0, v_{lon0} - \Delta v), \min(v_{limit}, v_{lon0} + \Delta v)], N), \quad (10)$$

where  $\Delta v$  determines the upper and lower bounds on longitudinal velocity, while  $N$  defines the granularity of the set. And  $\mathbb{V}_{lat}$  is similarly defined as follows:

$$\mathbb{V}_{lat} = \text{linspace}([-\min(v_{limit}, v_{lon0} + \Delta v), \min(v_{limit}, v_{lon0} + \Delta v)], N). \quad (11)$$

In order to reduce the sampling space size and computational load,  $\mathbb{A}_{lon}, \mathbb{A}_{lat}$  are set to  $\{0\}$ . And  $\mathbb{L}$  is set to  $\{0, \Delta d, d, -d\}$ , where  $d$  is the distance to the centerlines of neighbor lanes,  $\Delta d$  is the distance between the sampled endpoint of the human driving trajectory and the centerline of the lane. By incorporating  $\Delta d$ , the number of human-like generated trajectories can be increased, thereby enlarging the dataset size, which benefits the subsequent learning of the reward function. After calculation, the generated trajectory can be represented by a sequence of positions with regard to the  $lon$  and  $lat$  axis  $\sigma = \{(lon(0), lat(0)), \dots, (lon(\tau), lat(\tau))\}$ , which is then translated back to the Cartesian coordinate  $\sigma = \{(x(0), y(0)), \dots, (x(\tau), y(\tau))\}$ .

By utilizing a PID controller to track the generated trajectories  $\sigma^1, \sigma^2, \dots$  and real human trajectories  $\sigma^0$ , the corresponding segments of state-action pairs can be obtained. The reason for employing a controller to track real human trajectories is due to the limitation of available datasets, which do not contain the actual control actions (acceleration and steering angle) taken by human drivers. Instead, most datasets primarily provide trajectory data; therefore, it is necessary to use a controller to infer the missing control information. It is worth noting that there are some discrepancies between the control action inferred by the controller and that of real human drivers. Therefore, this article only explores the human-likeness of trajectories. The segments of state-action pairs produced by tracking real human trajectories are considered human driving segments and are labeled as preferred. For the segments generated by tracking generated trajectories, any segments that result in collisions with surrounding vehicles or driving off-road are discarded,

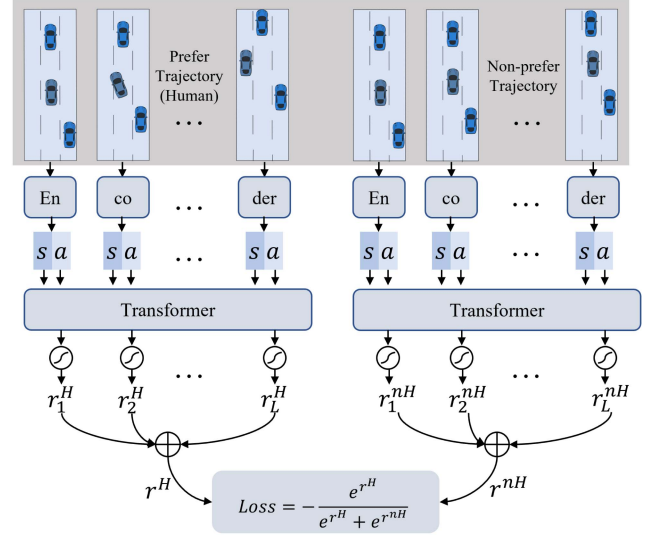


Fig. 3. Training process for the reward function.

while the non-colliding segments are assigned labels indicating whether they are preferred or non-preferred based on the human-likeness metric (final distance error (FDE), due to only exploring the human-likeness of trajectories). Human-like trajectories are considered preferred to ensure that subsequent reward functions possess human-like characteristics and capture the information about human decision-making processes. Ultimately, this method yields several segments of state-action pairs with preference labels, and each segment can be represented as  $\sigma = \{(s_0, a_0), \dots, (s_\tau, a_\tau)\}$ . Then, for a given initial state in a driving scenario, a preferred segment  $\sigma^0$  is paired with a corresponding non-preferred segment  $\sigma^1$  to form a trajectory pair  $(\sigma^0, \sigma^1, y)$ . These trajectory pairs constitute the elements of the overall dataset  $\mathcal{D} = \{(\sigma_0^0, \sigma_0^1, y_0), \dots\}$ , which is then used to train the reward function.

### C. Offline Preference Learning

With the availability of an offline dataset containing preference labels before RL training phase, it becomes feasible to learn a reward function through offline preference learning. The training process for the reward function is illustrated in Fig. 3. Firstly, we propose a self-attention based state encoding method to capture the interaction information between the ego vehicle and its surrounding vehicles. After encoding the state-action pairs, we train a transformer network through the offline preference learning framework. This enables the network to effectively model the human driving decision-making process, resulting in the final reward function used for RL training.

Since human driving is a sequential decision-making process, in order to capture temporal dependencies, the reward function, as the most concise description of the decision-making task, should be represented as a sequence modeling network. Motivated by the strong performance of transformers in modeling sequences, especially decision-making sequences [35], [36], we propose a transformer-based architecture to construct the reward

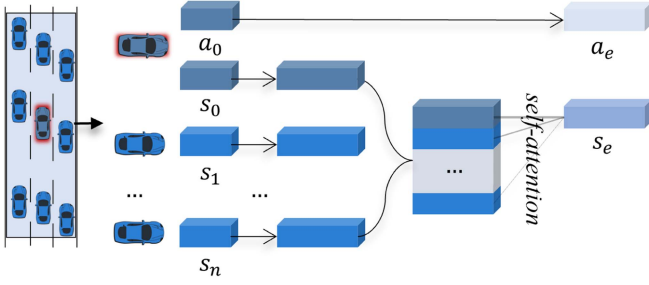


Fig. 4. The process of state-action encoding for reward function.

function for modeling decision sequences. To better conform to the MDP, we employ a GPT-2 architecture [37], which is an advanced transformer architecture with causally masked self-attention. This ensures that the reward obtained at timestep  $t$  is only related to the state-action at the current and preceding timesteps.

Since driving a vehicle involves interactions with surrounding vehicles, a strong encoder is essential for the reward function. The encoder captures the interactions between the ego vehicle and surrounding vehicles, encoding them into a state representation  $s_t$ , which can be subsequently utilized by the reward function. Human drivers exhibit varying levels of attention towards surrounding vehicles while driving, with a general tendency to allocate higher attention to vehicles that pose a higher risk of collision. Motivated by this fact, we propose a state encoding method based on a self-attention mechanism. Moreover, the self-attention mechanism can handle variable-length inputs, enabling the state encoder to process the varying number of surrounding vehicles during driving. The structure of state-action encoder consists of linear network layers and the self-attention mechanism. The encoding process is illustrated in Fig. 4. For the state encoder, the input is the self and surrounding vehicles' states  $\{s_0, s_1, \dots\}$ . The state  $s$  includes the position, length, width, speed, and heading angle of the vehicle. And surrounding vehicles refer to the nearest vehicles around the ego vehicle (includes the leading vehicle and following vehicle in the same lane, as well as parallel vehicles, leading vehicles, and following vehicles in adjacent lanes, within a distance of 50 meters from the ego vehicle). The states of the ego vehicle  $s_0$  and the surrounding vehicles  $\{s_1 \dots s_n\}$  are first passed through linear network layers  $\ell$ , then through self-attention  $\mathcal{A}$ . The first output from the self-attention is taken as the final encoded state  $s_e$  and the calculation process is as follows:

$$s_e = \mathcal{A}(\ell(s_0), \ell(s_1), \dots, \ell(s_n))[0]. \quad (12)$$

For action encoder, the control variables  $a_0$  of the ego vehicle (acceleration and steering angle) are passed through the linear network layers to obtain the final encoded action  $a_e$ .

A trajectory segment of length  $L$ , after encoding, produces a state-actions input  $\sigma = \{s_e^1, a_e^1, \dots, s_e^L, a_e^L\}$  of length  $2L$ . A shared positional embedding is learned and appended to each input, where the encoded state and encoded action at the same timestep utilize the same positional embedding. The input, appended with positional embedding, is fed into the transformer

network, which produces output embeddings. These output embeddings are then fed into a linear layer to generate the rewards for each timestep. To ensure the reward value output  $r$  are bounded between (0,1), which can facilitate easier training during subsequent RL, the linear layer's output is processed through a sigmoid activation to obtain the final scalar reward for each timestep.

The training process of the reward function can be formulated under the framework of offline preference learning. The loss function is designed based on (5):

$$\mathcal{L}(\psi) = -\mathbb{E}_{(\sigma^H, \sigma^{nH}) \sim \mathcal{D}} \left[ \log \frac{e^{\sum_t r_t^H}}{e^{\sum_t r_t^H} + e^{\sum_t r_t^{nH}}} \right], \quad (13)$$

where  $\sigma^H$  denotes the trajectory segment that is human-like with preference label, and  $\sigma^{nH}$  denotes the trajectory segment with non-preference label. The training objective of the reward function is to maximize the probability of selecting human-like trajectories as preferred trajectories, which is equal to minimizing the loss function (13).

#### D. Reinforcement Learning

By utilizing human driving data, a reward function can be learned. The learned reward function are then incorporated into a RL framework to guide the agent towards human-like driving policies. To ensure training stability, we utilize the Proximal Policy Optimization (PPO) RL framework to train the final policy network.

PPO [38] is a popular policy gradient algorithm in deep RL and it demonstrates excellent performance in addressing complex control problems. Especially, the PPO algorithm is well-suited for RL problems where the reward function is learned from data rather than handcrafted, due to PPO's stability during training [39]. Motivated by this fact, we utilize it to train our final driving decision network. By leveraging PPO's sample efficiency and robustness to inevitable noise in the data-driven reward function, we can develop an autonomous driving policy guided by human driving knowledge through the learned reward function. The workflow of our algorithm (based on the framework of the PPO algorithm) can be shown in Fig. 5. The workflow starts with the agent sampling experiences from the environment through interaction. The agent can obtain rewards by assessing states and corresponding actions using the learned reward function. These sampled experiences are then leveraged to estimate the advantage function via generalized advantage estimation (GAE), refining the approximation of the advantage. GAE takes into account the rewards and values of multiple timesteps, which can result in a more accurate estimate of the advantage function and improve the overall performance and stability of the RL process [40]. Based on the data from the experience buffer, the loss functions for the value network and policy network can be computed, namely the MSE loss (14) and PPO loss (15), where  $V_\phi(s_t)$  is the value of state  $s_t$ ,  $\hat{A}_t$  is the advantage, and  $r_t(\theta)$  is the KL divergence. These losses are further utilized to optimize the value network and policy network through gradient descent. After several rounds of training, the final driving policy network can be obtained. To stabilize the RL

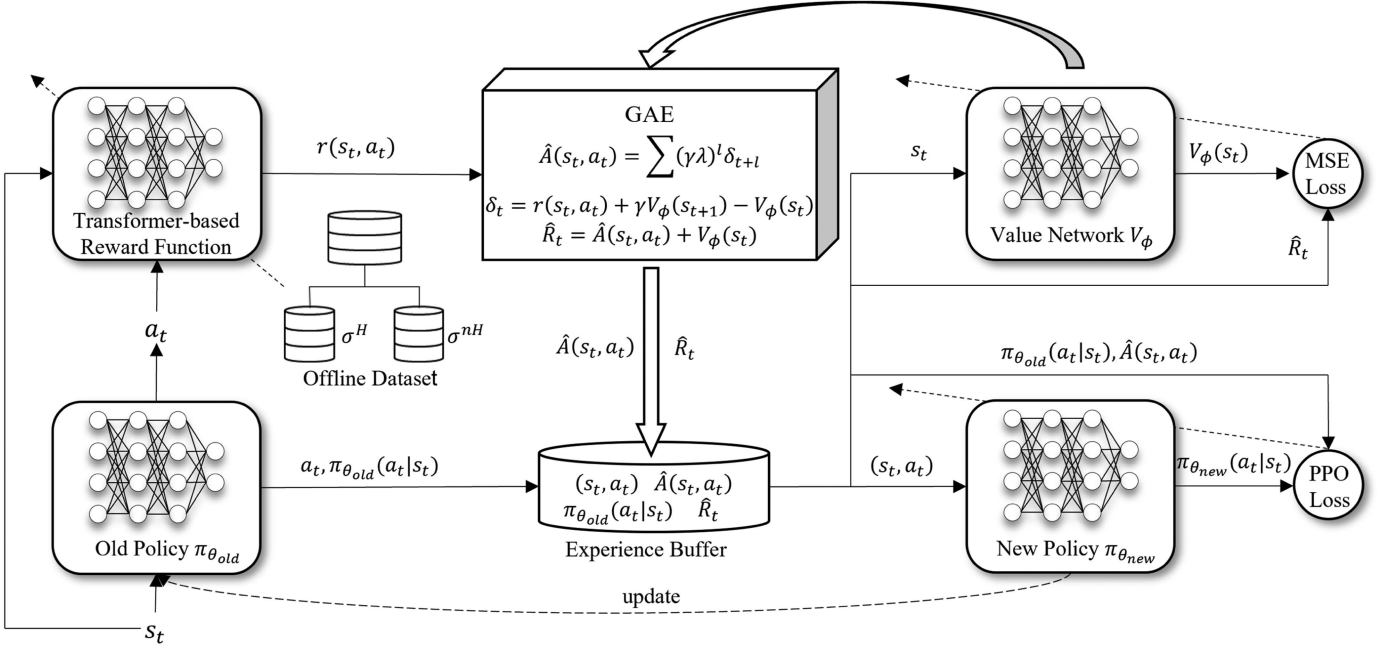


Fig. 5. The workflow of HGRL algorithm.

training, we employ reward scaling and clipping tricks, which normalize rewards by dividing them by the standard deviation of a rolling discounted sum of rewards (without subtracting and re-adding the mean) and clip rewards to a preset range [41].

$$\mathcal{L}_\phi = \mathbb{E} \left[ \left( V_\phi(s_t) - \hat{R}_t \right)^2 \right]. \quad (14)$$

$$\begin{aligned} \mathcal{L}_\theta^{CLIP} &= \mathbb{E} [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)] \\ \text{s.t. } r_t(\theta) &= \frac{\pi_{\theta_{new}}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}. \end{aligned} \quad (15)$$

## V. EXPERIMENTAL SETUP

### A. Highway Simulator

Highway environments are considered to be the first environments where fully automated driving could potentially be achieved [42]. An increasing number of research studies and companies are focusing on automated driving in highway environments. From a practical standpoint, this article selects the highway environment as the simulation setting for validating the performance of the HGRL algorithm. The goal of the HGRL algorithm is to realize human-like and safe driving decisions. Thus, in the simulation environment, surrounding vehicles should follow real human driving trajectories in order to reflect driving scenarios encountered during human driving as realistically as possible. Regrettably, the surrounding vehicle behaviors in most existing simulators are generated based on traffic flow models, such as in HighwayEnv [43]. They cannot accurately reflect the real-world driving scenarios encountered by humans.

To solve this problem, we build our own simulator on top of the highway platform provided by Huang et al.'s improved

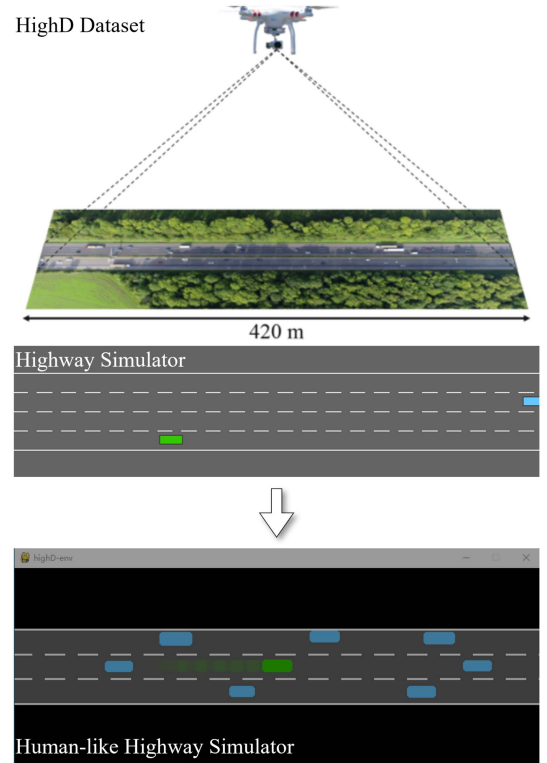


Fig. 6. The construction process of human-like highway simulator.

simulator based on HighwayEnv [26]. We incorporate the HighD dataset [44], which consists of naturalistic vehicle trajectory data recorded on German highways, into our simulator. The construction process of our human-like highway simulator is shown in Fig. 6. The highway simulator incorporates surrounding vehicle trajectories generated based on real-world scenarios, providing



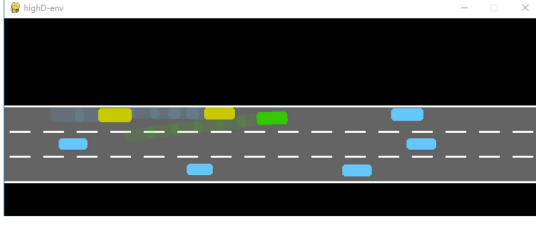


Fig. 7. Demonstration of the interaction between the ego vehicle and surrounding vehicles when the ego vehicle greatly deviates from human driving behavior.

the potential to train and validate human-like driving policies. During the training of the ego vehicle's policy in the simulator, it is inevitable that surrounding vehicles will encounter behaviors from the ego vehicle that deviate significantly from real human driving. In order to anticipate the reactions of surrounding vehicles to these deviations by the ego vehicle and make the simulation environment more realistic, we incorporated an intelligent driver model (IDM) into the surrounding vehicle models. IDM is a parametric car-following model that uses mathematical equations to simulate a driver's desired tradeoff between maintaining speed and keeping a safe distance from the preceding vehicle, which is popular for highway traffic simulation [26]. When the distance between the surrounding vehicles (following vehicles of the ego vehicle) and the ego vehicle is less than a set value, the surrounding vehicles will not follow the original real human trajectory but will instead take the control action output by the IDM model. As shown in Fig. 7, when the ego vehicle (the green vehicle in the figure) makes a lane-change maneuver (greatly deviating from realistic human behavior), the following vehicles in the same lane (the yellow vehicles in the figure) no longer follow the original real human trajectory but instead brake according to the IDM.

Since the HighD dataset only records naturalistic human driving trajectories over a 420-meter road section and the vehicle states at the start and end points have some errors, only trajectories that started or ended at non-edge positions can be used. Also, we select vehicle trajectories with mean speeds all greater than 20 m/s, some even exceeding 30 m/s, which reflects normal traffic flow and normal driving behavior on highways. Therefore, the duration of each driving scenario  $\tau$  is set to 5 seconds. The sampling frequency of the HighD dataset is 25 Hz. To remain consistent, the decision frequency of the algorithm is also set to 25 Hz. Due to the increased number of decision steps (total 125 steps), this also increases the difficulty of the control task to some extent. As highway driving primarily involves two behaviors, car-following and lane-changing, this article designs two classes of driving scenarios for research: in one class of scenarios, humans only behave car-following behavior, while in the other class of scenarios, humans perform both car-following and lane-changing behaviors. In both classes of scenarios, the algorithm outputs both acceleration and steering angle control signals to better verify the performance of the proposed algorithm.

## B. HGRL Settings

1) *Reachable State Space for Trajectory Sample*: Reachable state space  $\mathbb{S}$  determines the size of the dataset for offline

preference learning given a fixed number of naturalistic human driving trajectories. However, an excessively large state space significantly lengthens the time of trajectory sample. We balanced considerations of state space coverage and sampling time efficiency to determine the following parameters of reachable state space  $\mathbb{S}$ :

$$\begin{aligned} \mathbb{V}_{lon} &= \text{linspace}([\max(0, v_{lon0} - 5), \\ &\quad \min(v_{limit}, v_{lon0} + 5)], 10); \\ \mathbb{V}_{lat} &= \text{linspace}([-v_{lon0} - 5, v_{lon0} + 5], \\ &\quad 2 \times \text{int}(v_{lon0} + 5) + 1), \text{if } lat_e \neq (0 \text{ or } \Delta d), \\ &\quad \text{else } \{0\}; \\ \mathbb{A}_{lon} &= \mathbb{A}_{lat} = \{0\}; \\ \mathbb{L} &= \{0, \Delta d, d, -d\}. \end{aligned} \quad (16)$$

Having determined the set of reachable state space, we can exhaustively enumerate the states in the set and utilize polynomials to generate trajectories. Similarly, we use polynomials to generate real human trajectories based on real human targets. By using a PID controller to follow these trajectories, we obtain segments of state-action pairs. For segments generated by tracking sampled trajectories, if their FDE is less than or equal to the average of this metric for segments generated by tracking real human driving trajectories, they are also regarded as real human driving trajectories and labeled as preferred. We believe this processing can improve the generalizability of the reward function when training it later, since it incorporates some high-quality non-human driving trajectories into the set of preferred trajectories. Additionally, it also increases the dataset size, enhancing the performance of the trained reward function. For each class of driving scenario, we randomly selected trajectories from 40 drivers recorded in the HighD dataset to generate the training set, 10 drivers for the validation set, and 10 drivers for the test set.

2) *Preference Learning Settings for Reward Function Learning*: After building the dataset, we can utilize it to train the reward function, and the training process can be shown in Fig. 3. The reward function is implemented in PyTorch and trained using the AdamW [45] optimizer to alleviate overfitting with the help of weight decay. And it runs on an NVIDIA RTX 3080 GPU. The reward function is trained using the training set, and the final reward function used for subsequent RL is the one that achieves the lowest loss on the validation set. Due to the fact that the dataset does not include collision trajectories, to further enhance the reward function's ability to evaluate the dangerous behavior of collisions, we add a collision penalty to the trained reward function. If the agent experiences a collision, we apply a penalty of  $-1$  instead of using the trained reward function  $r$  for evaluation at the timestep of collision. The reward function  $\hat{r}$  with the collision penalty is shown below:

$$\hat{r}_t(s, a) = \begin{cases} r_t(s, a), & \text{if not collision} \\ -1, & \text{if collision} \end{cases}. \quad (17)$$

3) *PPO Settings for Reinforcement Learning*: After training the reward function, we incorporated it into a PPO framework to train the final policy network, as shown in Fig. 5. In the



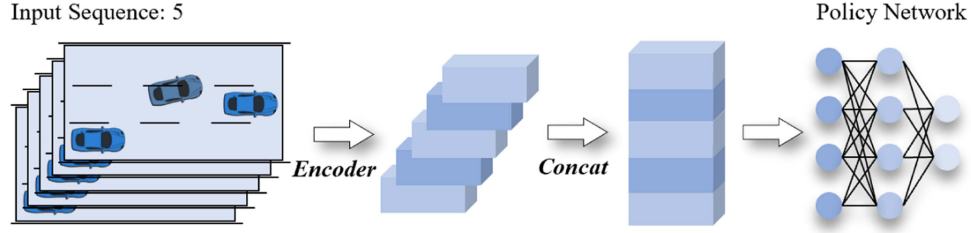


Fig. 8. The input structure for the policy network.

policy network, we utilized an encoder network based on self-attention with the same structure as the reward function to enable capturing the information of interactions between the self and surrounding vehicles. Additionally, human driving decisions depend on information about self and surrounding vehicles from previous timesteps. To enable the policy network to capture historical sequence information, the input to the policy network includes encoded states from the current and previous timesteps (total of 5 timesteps), which are concatenated and then fed into the policy network, as shown in Fig. 8. The input to the value network is identical to the input of the policy network, consisting of the concatenated encoded states from the current and previous 4 timesteps. Similarly, RL training is implemented in PyTorch and runs on an NVIDIA RTX 3080 GPU, using the Adam optimizer.

### C. Comparison Baselines

Some RL, IRL and IL methods are implemented as baselines to benchmark the performance of the proposed method. They are listed as follows.

1) *PPO-FBR*: PPO is a state-of-the-art RL algorithm, especially in the task of continuous control. To achieve human-like decision-making, we manually designed a FDE-based reward function (FBR) for PPO training as a baseline, named PPO-FBR. This reward function guides the policy network towards human-likeness by incorporating the FDE between the trajectory generated by the vehicle under the control of the policy network over a specified number of timesteps (125 timesteps in the task) and the real human driving trajectory under the same driving scenarios. Specifically, the reward function takes the form:

$$r_{FBR}(t) = \begin{cases} \frac{1}{N}, & \text{if not collision} \\ -1, & \text{if collision} \\ \max(e^{2.5-FDE}, 1), & \text{if } t = N \text{ and not collision} \end{cases}, \quad (18)$$

where  $N$  is the total timesteps for the task. The safety component of the reward function comprise of penalties for collisions and rewards for each non-colliding step, while the human-likeness component,  $\max(e^{2.5-FDE}, 1)$ , rewards trajectory similarity to human drivers over time. This composite reward function aims to guide the agent to avoid collisions and closely match real human trajectories under the same driving scenario. And the network structure employed in the PPO-FBR algorithm aligns with that of the RL segment of our proposed framework, which also

includes encoder networks based on self-attention and inputs state information across multiple timesteps.

2) *SACfD*: SACfD is a RL algorithm with human demonstrations, learning driving decision strategies through human demonstrations and interactions with the environment. This algorithm is inspired by DQfD (Deep Q-learning from Demonstrations) [21], it incorporates SAC (Soft Actor-Critic), a state-of-the-art RL algorithm for continuous action spaces, enabling its application to the driving decision-making task discussed in this paper. It begins with offline training of the SAC algorithm using human demonstration data, followed by online training where both human demonstration data and data generated from interactions between the SAC agent and the environment are fed into the replay buffer to train the SAC algorithm. The reward function setup is consistent with that of the PPO-FBR algorithm, and the structures of the policy network and value network are also in line with PPO-FBR (and therefore consistent with HGRL).

3) *GAIL*: GAIL is a state-of-the-art IRL algorithm. GAIL imitates expert demonstration policies through adversarial learning between a discriminator that provides a surrogate reward and a RL agent that utilizes the given surrogate reward from the discriminator to train the policy network. To capture sequential information and information about the interactions between the ego vehicle and surrounding vehicles, the input states match those of the RL policy network in HGRL and also use an encoder network based on self-attention.

4) *Behavioral Cloning (BC)*: BC is a widely used method of IL in autonomous driving [46]. BC directly learns the expert's demonstrated actions in given states through supervised learning. Similarly, the input states match those of the RL policy network in HGRL and also use an encoder network based on self-attention to capture sequential and spatial information.

## VI. RESULTS AND DISCUSSIONS

### A. Metrics

Since the reward functions employed in our proposed method and other baseline methods are different, the reward values, which are commonly used as a performance metric in RL, cannot be directly compared across these algorithms. Therefore, it is necessary to design some metrics that can effectively evaluate the performance of different algorithms. As the article aims for obtaining a safe and human-like driving decision-making system, we use the widely adopted task success ratio to evaluate

safety and the mean FDE metric, also commonly used, to evaluate the human-likeness of the learned policies. Furthermore, to comprehensively assess the safety and human-likeness of the learned policies, we introduce a novel metric called the Success-Human-likeness index (SH index).

The task success ratio is determined by dividing the number of successful episodes within a span of 8192 steps by the total number of episodes experienced during those 8192 steps. And the failure conditions include going off the road and colliding with surrounding vehicles. The mean FDE is the average of the FDE between the generated trajectory and the human trajectory for successful episodes out of 8192 steps. If no successful episodes are achieved within 8192 steps, the mean FDE is set to 50. The SH index is a comprehensive metric that combines the two aforementioned metrics to represent the overall performance of the learned policy in terms of safety and human-likeness. It provides a measure of how well the policy balances both safety and human-likeness, formulated as:

$$SH = SR \times \max(e^{2.5-FDE}, 1), \quad (19)$$

where  $SR$  indicates the task success ratio, and 2.5 is the maximum FDE allowed for the PID controller to track human trajectories in the phase of trajectory sample. Thus we consider the algorithm that output control values yielding driving trajectories with FDE less than or equal to 2.5 compared to real human trajectories demonstrates excellent human-likeness. And the metric is similar to the reward function of the PPO-FBR algorithm (a benchmark). Thus, the metric is the optimization direction of the PPO-FBR algorithm. If our proposed algorithm can outperform the PPO-FBR algorithm on this metric, it can demonstrate the superiority of our algorithm. (20) shows the partial derivative of SH index, from which we can see that the task success ratio has a greater influence on SH index than FDE. To some extent, this indicates the metric prioritizes safety over human-likeness, which aligns with mainstream values.

$$\begin{aligned} & \text{if } FDE \geq 2.5 : \\ & \begin{cases} \frac{\partial SH}{\partial SR} = e^{2.5-FDE} \\ \frac{\partial SH}{\partial FDE} = -SR \times e^{2.5-FDE} \end{cases} \end{aligned} \quad (20)$$

## B. Learning Result

Our proposed algorithm consists of two stages during the learning process. Firstly, offline preference learning is used to learn the reward function. Secondly, RL is used to learn the final policy network. In the reward function learning stage, we train reward functions based on transformer, Multi-Layer Perceptron (MLP), and Long Short-Term Memory network (LSTM), and compared their effectiveness in guiding the learning of the policy network. In the policy network learning stage, we compared the performance of the policy network learned by our proposed algorithm with that of the baseline algorithms.

1) *Reward Function Learning Result:* For each class of driving scenarios, we train a reward function using the training set. We then fine-tune the reward function based on the loss on the validation set. And the optimal hyperparameters are shown in Table I. We also train reward functions with different structures

TABLE I  
OPTIMAL HYPERPARAMETERS USED FOR REWARD FUNCTION

| Hyperparameters                       | Value         |                                 |
|---------------------------------------|---------------|---------------------------------|
|                                       | Car-following | Lane-changing and Car-following |
| Hidden Units                          | 64            | 256                             |
| Number of Transformer Layer           | 1             | 1                               |
| Number of Heads for Transformer Layer | 4             | 8                               |
| Number of Heads for Encoder           | 4             | 8                               |
| Batch Size                            | 128           | 128                             |
| Learning Rate                         | 0.0001        | 0.0001                          |
| $(\beta_1, \beta_2)$                  | (0.9, 0.99)   | (0.9, 0.99)                     |
| Weight Decay                          | 0.0001        | 0.0001                          |

for comparison, including a reward function based on MLP and a reward function based on LSTM. The obtained reward functions are then incorporated into subsequent RL training, and the learning results are shown in Fig. 9. To smooth the learning curve, we employ a convolutional approach. Every 8192 steps, we take the average of the metric values over a kernel size of  $8192 \times 30$  steps. In the class of car-following scenarios, the RL agent guided by the transformer-based reward function achieves optimal performance across all metrics. The RL agent guided by the LSTM-based reward function outperforms the MLP-based reward function in terms of the success ratio metric, indicating better decision safety. However, it performs worse in the FDE metric, indicating lower decision human-likeness, compared to the MLP-based reward function. In the class of car-following and lane-changing scenarios, although the RL agent guided by the transformer-based reward function has slightly lower success ratio compared to the LSTM-based reward function, it exhibited higher performance in the comprehensive metric (SH index). The experimental results affirm that the transformer-based reward function used in our proposed algorithm effectively guides the learning direction of the RL agent, overall outperforming those based on LSTM and MLP. This can be attributed to the superior sequence modeling capability of the transformer, while the limited size of the training set may cause the relatively small performance gap between the transformer-based and LSTM-based reward functions in the class of car-following and lane-changing scenarios. Additionally, the successful application of diverse neural network architectures for reward functions within our framework substantiates the effectiveness of our framework.

2) *Reinforcement Learning Result:* The proposed algorithm HGRL (transformer-based reward function) and baseline algorithms are trained in the designed two classes of driving scenarios. The expert trajectories provided to the reward function in HGRL, the SACfD, the GAIL, and the BC are from 40 drivers' trajectories (training set) derived from the HighD dataset. Fig. 10 represents the learning results of HGRL and the baseline algorithms. To smooth the learning curve, we employ the convolutional approach similarly. BC algorithm is not shown in Fig. 10 as it is unable to successfully complete the 125 steps in every episode due to insufficient data from just 40 drivers' driving behaviors. It illustrates the high data requirements of imitation learning.

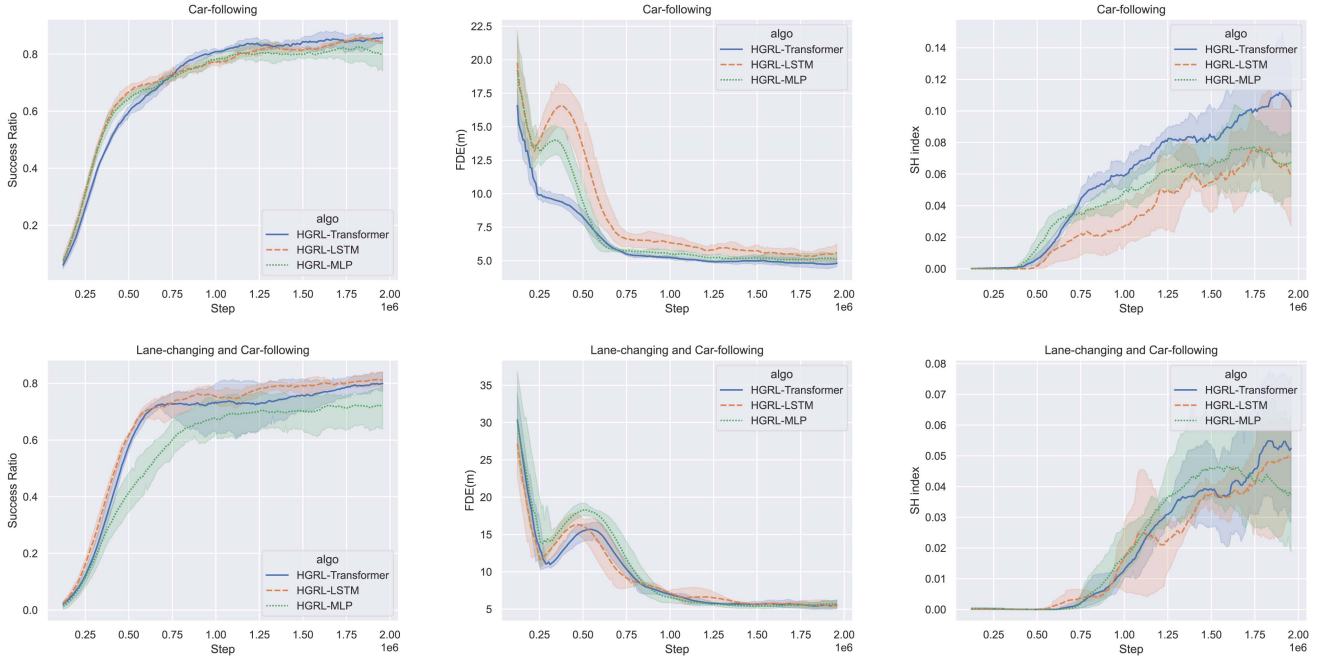


Fig. 9. The learning result guided by different reward functions.

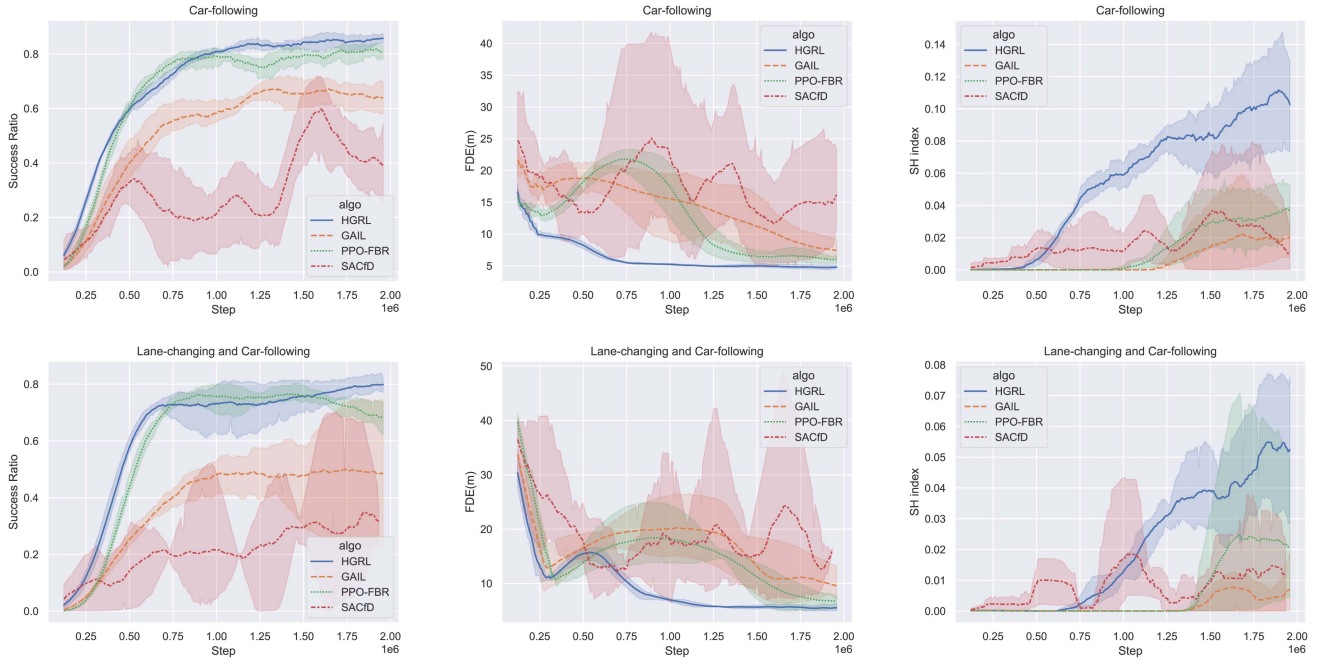


Fig. 10. The learning result of HGRL, GAIL, PPO-FBR and SACfD (BC can not successfully complete the task).

Our proposed HGRL algorithm outperforms other baseline algorithms in terms of both decision safety (success ratio) and decision human-likeness (FDE) in both classes of driving scenarios. The HGRL algorithm also achieves the highest performance in the comprehensive metric (SH index). In contrast, the GAIL algorithm is sensitive to the seed during training and exhibits an unstable training process, which is one of the main drawbacks of the algorithm [47]. The SACfD is also sensitive to the seed during training and does not perform well. It should

be noted that the SACfD training process is not stable, and when applying convolutional smoothing to the learning curves, some local optima in the success ratio and FDE metrics are smoothed out. Nonetheless, these smoothed local optima are still reflected in the SH index, which is derived from the success ratio and FDE. During the initial phase of training, this index is superior to those of the other algorithms. And it might be due to the offline training with human demonstration data, which provides a better initial performance. The underperformance of



the SACfD algorithm might be attributed to the higher reward function sensitivity of the SAC algorithm compared to the PPO algorithm. This outcome suggests that manually designed reward functions are challenging to effectively guide RL agents' learning, even with the assistance of human demonstration data. The PPO-FBR algorithm, while capable of achieving relatively good performance, falls short compared to the HGRL algorithm across all metrics. Notably, the FDE curve of the PPO-FBR algorithm shows a significant increase before decreasing. This trend indicates that the handcrafted reward function fails to effectively guide the policy network towards a direction that balances both safety and human-likeness. Particularly in the class of lane-changing and car-following driving scenarios, as the FDE decreases, the success ratio also decreases, further highlighting this issue. In contrast, the HGRL algorithm exhibits a different trend in both classes of scenarios. Although there is a slight initial increase in FDE for the lane-changing and car-following scenarios, the overall trend shows a decrease in FDE as the success ratio increases. Importantly, there is no observed decrease in the success ratio as the FDE decreases. In contrast, there is actually an increase in the success ratio as the FDE decreases. This indicates that HGRL can accurately model human driving decision-making processes, derive an accurate reward function, and effectively guide the policy network to learn in a direction that balances both human-likeness and safety. Moreover, the HGRL algorithm demonstrates the lowest sensitivity to training seeds in the FDE metric. This stability across different seeds implies that the guiding role of the learned reward function in HGRL is less affected by initial conditions or random variations.

### C. Evaluation Result

To further evaluate the performance of our proposed algorithm, we conduct two experiments. The first experiment validates the generalization capabilities of the learned reward functions across different drivers. The second experiment assesses the effectiveness of policy networks trained through HGRL in longer decision tasks.

1) *Evaluating Performance of Reward Function Across Different Drivers*: To validate the generalization capabilities of the learned reward functions (transformer-based, LSTM-based and MLP-based) across different drivers, we introduce new driving scenarios based on the driving trajectories of 10 drivers from the test set for both classes of driving scenarios. We utilize the learned reward functions (trained on the training set, excluding the driving data of the 10 drivers) and RL, specifically our proposed HGRL algorithm, to train the policy network and observe its training performance in these new scenarios. This allowed us to assess whether the reward functions can guide the direction of the policy network well in driving scenarios it had not encountered during training, thus validating the generalization capabilities of the reward functions across different drivers. In order to compare the generalization capabilities of other algorithms across different drivers during the training process, we also trained the GAIL and BC algorithms. It is important to note that the PPO-FBR and SACfD algorithm, which

require trajectories from newly introduced drivers to calculate reward during training, are not included in the comparison as they do not possess inherent generalization capabilities across different drivers during training. Similarly, the GAIL and BC algorithms are trained using only the driving data from the training set (excluding the data from the 10 newly introduced drivers).

Fig. 11 shows the learning result of HGRLs and GAIL across different drivers (BC can not successfully complete the task). We also use the convolutional approach to smooth the learning curve. From Fig. 11, it can be observed that HGRL, HGRL-LSTM, and HGRL-MLP outperform GAIL and BC in all three metrics, demonstrating the superiority of our proposed method. Despite a slight decline in performance, the policy networks guided by learned reward functions still significantly outperform the GAIL and BC algorithms. The results indicate that the learned reward functions effectively guide the policy network to achieve safe and human-like learning, even in new driving scenarios encountered by different drivers that are not present in the training set for reward function training. Among the approaches, the transformer-based reward function exhibits the best performance, which can be attributed to the advantages of the transformer in sequence modeling.

2) *Evaluation Performance in Extended Decision Tasks*: In order to assess the effectiveness of policy networks trained through HGRL (transformer-based reward function) in longer decision tasks, we conduct an experiment involving extended decision tasks with 200 steps (a 60% increase from the previous 125-step tasks). The objective is to test the performance of the HGRL-trained policy networks in more challenging and time-intensive decision tasks. For both classes of driving scenarios, we design 30 new 200-step driving decision tasks (driving scenarios). We select policy networks trained within a fixed number of training steps (3.2 million) to assess their performance in these tasks. And the result is shown in Table II. As the total step number of task increases, the FDE tends to increase. However, the increase in FDE typically remains within the range of the increase in decision steps. In other words, the magnitude of the FDE increase does not surpass the magnitude of the increase in task steps for trained policy networks. On the other hand, the success ratio even increases, which may be explained by the fact that during the training phase, actions are randomly sampled based on the mean and variance outputs generated by the network, while during the testing phase, actions are determined deterministically based on the mean output generated by the network. The high success ratio indicates that the policy network is capable of handling the extended decision task effectively. Therefore, we believe that the policy networks trained by HGRL algorithm exhibit good performance in extended decision tasks.

### D. Discussions

One of the challenge of RL-based driving decision-making system lies in the design of the reward function. Manually designed reward functions may not effectively guide the learning direction of RL and may even lead to unexpected ways for the RL agent to achieve high reward returns. In our proposed algorithm,

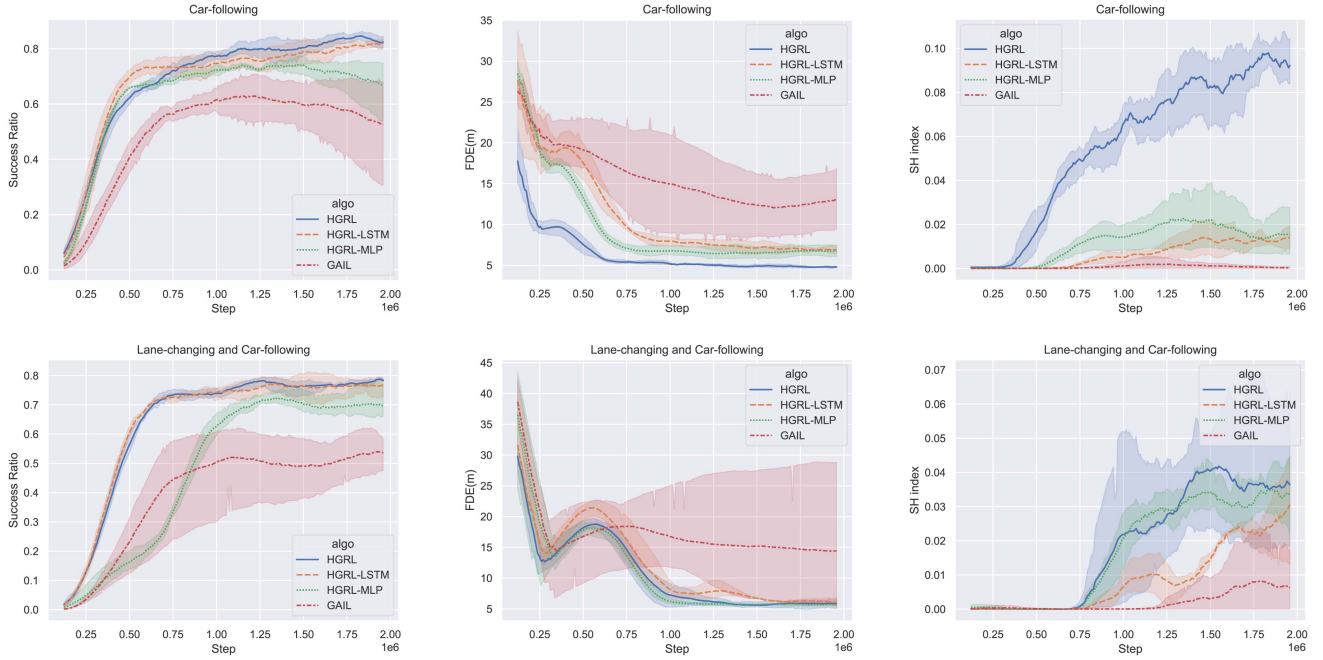


Fig. 11. The learning result of HGRLs and GAIL across different drivers (BC can not successfully complete the task).

TABLE II  
TESTING RESULT IN EXTENDED DECISION TASKS

| Type of Driving Scenario        | 125-step Task (train) |       | 200-step Task (test) |       |
|---------------------------------|-----------------------|-------|----------------------|-------|
|                                 | Success Ratio         | FDE   | Success Ratio        | FDE   |
| Car-following                   | 95.58%                | 3.50m | 100%                 | 4.54m |
| Lane-changing and Car-following | 90.00%                | 3.96m | 93.33%               | 4.84m |

we leverage a comparison between human driving data and generated potential trajectories to capture information about the human decision-making process and model human decision process, ultimately obtaining a reward function. Experimental results demonstrate that the learned reward function can effectively guide the RL agent's learning direction, leading to more human-like and safer decision-making compared to existing state-of-the-art algorithms. Some details and videos about our work can be found at the <https://github.com/pHis2/HGRL>.

However, some weakness of the proposed method should be acknowledged. Due to the use of a simple PID controller to track human trajectories, the resulting state-action pairs may not accurately reflect human driving actions. This introduces some discrepancies in the information captured by the reward function, limiting our assessment of human-likeness to the level of driving trajectories rather than delving into the level of driving actions. It also reduces the quality of the dataset to some extent, which hinders the further improvement of algorithm performance. Additionally, the driving scenarios currently employed are focused on highway environments, which are relatively limited in diversity. Thus, in the future, more complex driver models will be explored to track trajectories, aiming to

generate state-action pairs that better align with real human driving behavior. This will enhance the accuracy of modeling the human driving decision-making process, leading to a more precise reward function. Additionally, our method will be further explored in a wider range of complex driving scenarios, including roundabouts, intersections, and other road designs.

## VII. CONCLUSION

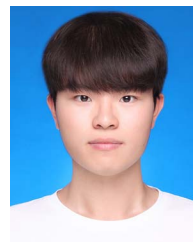
In this article, a RL-based decision-making framework for autonomous driving is proposed, utilizing human driving data to learn the reward function. By comparing human trajectories with generated feasible trajectories, a transformer network is trained within the offline preference learning framework to model the human driving decision-making process and obtain a reward function for RL. The learned reward function is then integrated into the RL framework to obtain a driving decision-making network. In simulated highway environments, the decision-making network obtained through our method outperforms baseline algorithms in terms of both human-likeness and safety. The learned decision-making network also performs well in extended driving decision tasks. Experimental results demonstrate that our proposed method can obviate the requirement for manual design of sophisticated reward functions in RL-based autonomous driving decision-making systems by leveraging the decision-making information embedded in human driving data to learn the reward function.

## ACKNOWLEDGMENT

The authors are immensely grateful to Dr. He Xiangkun for his valuable advice on both the manuscript and experimental work of this article.

## REFERENCES

- [1] L. Chen et al., "Milestones in autonomous driving and intelligent vehicles: Survey of surveys," *IEEE Trans. Intell. Veh.*, vol. 8, no. 2, pp. 1046–1056, Feb. 2023.
- [2] L. Chen et al., "Milestones in autonomous driving and intelligent vehicles—Part 1: Control, computing system design, communication, hd map, testing, and human behaviors," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 53, no. 9, pp. 5831–5847, Sep. 2023.
- [3] X. Tang et al., "Prediction-uncertainty-aware decision-making for autonomous vehicles," *IEEE Trans. Intell. Veh.*, vol. 7, no. 4, pp. 849–862, Dec. 2022.
- [4] P. Hang, C. Lv, Y. Xing, C. Huang, and Z. Hu, "Human-like decision making for autonomous driving: A noncooperative game theoretic approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2076–2087, Apr. 2021.
- [5] X. He, W. Huang, and C. Lv, "Toward trustworthy decision-making for autonomous vehicles: A robust reinforcement learning approach with safety guarantees," *Engineering*, vol. 33, pp. 77–89, 2023.
- [6] B. R. Kiran et al., "Deep reinforcement learning for autonomous driving: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 4909–4926, Jun. 2022.
- [7] P. R. Wurman et al., "Outracing champion gran turismo drivers with deep reinforcement learning," *Nature*, vol. 602, no. 7896, pp. 223–228, 2022.
- [8] X. He et al., "Fear-neuro-inspired reinforcement learning for safe autonomous driving," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 1, pp. 267–279, Jan. 2024.
- [9] J. Lu, L. Li, and F.-Y. Wang, "Event-triggered parallel control using deep reinforcement learning with application to comfortable autonomous driving," *IEEE Trans. Intell. Veh.*, vol. 9, no. 3, pp. 4470–4479, Mar. 2024.
- [10] Y. Qu et al., "RL-driven MPPI: Accelerating online control laws calculation with offline policy," *IEEE Trans. Intell. Veh.*, vol. 9, no. 2, pp. 3605–3616, Feb. 2024.
- [11] K. Lee, L. M. Smith, and P. Abbeel, "Pebble: Feedback-efficient interactive reinforcement learning via relabeling experience and unsupervised pre-training," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 6152–6163.
- [12] C. Kim, J. Park, J. Shin, H. Lee, P. Abbeel, and K. Lee, "Preference transformer: Modeling human preferences using transformers for RL," in *Proc. 11th Int. Conf. Learn. Representations*, 2022.
- [13] W. Li, F. Qiu, L. Li, Y. Zhang, and K. Wang, "Simulation of vehicle interaction behavior in merging scenarios: A deep maximum entropy-inverse reinforcement learning method combined with game theory," *IEEE Trans. Intell. Veh.*, vol. 9, no. 1, pp. 1079–1093, Jan. 2024.
- [14] S. Adams, T. Cody, and P. A. Beling, "A survey of inverse reinforcement learning," *Artif. Intell. Rev.*, vol. 55, no. 6, pp. 4307–4346, 2022.
- [15] Z. Huang, J. Wu, and C. Lv, "Efficient deep reinforcement learning with imitative expert priors for autonomous driving," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 10, pp. 7391–7403, Oct. 2023.
- [16] Y. Huang, Y. Gu, K. Yuan, S. Yang, T. Liu, and H. Chen, "Human knowledge enhanced reinforcement learning for mandatory lane-change of autonomous vehicles in congested traffic," *IEEE Trans. Intell. Veh.*, vol. 9, no. 2, pp. 3509–3519, Feb. 2024.
- [17] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30.
- [18] S. Casper et al., "Open problems and fundamental limitations of reinforcement learning from human feedback," 2023, *arXiv:2307.15217*.
- [19] S. Arora and P. Doshi, "A survey of inverse reinforcement learning: Challenges, methods and progress," *Artif. Intell.*, vol. 297, 2021, Art. no. 103500.
- [20] X. Liang, T. Wang, L. Yang, and E. Xing, "CIRL: Controllable imitative reinforcement learning for vision-based self-driving," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 584–599.
- [21] T. Hester et al., "Deep Q-learning from demonstrations," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 3223–3230.
- [22] H. Liu, Z. Huang, J. Wu, and C. Lv, "Improved deep reinforcement learning with expert demonstrations for urban autonomous driving," in *Proc. IEEE Intell. Veh. Symp.*, 2022, pp. 921–928.
- [23] J. Wu, Z. Huang, Z. Hu, and C. Lv, "Toward human-in-the-loop AI: Enhancing deep reinforcement learning via real-time human guidance for autonomous driving," *Engineering*, vol. 21, pp. 75–91, 2023.
- [24] L. Le Mero, D. Yi, M. Dianati, and A. Mouzakitis, "A survey on imitation learning techniques for end-to-end autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 14128–14147, Sep. 2022.
- [25] S. Rosbach, V. James, S. Großjohann, S. Homocanu, and S. Roth, "Driving with style: Inverse reinforcement learning in general-purpose planning for automated driving," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 2658–2665.
- [26] Z. Huang, J. Wu, and C. Lv, "Driving behavior modeling using naturalistic human driving data with inverse reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 10239–10251, Aug. 2022.
- [27] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, vol. 29.
- [28] P. Wang, D. Liu, J. Chen, H. Li, and C.-Y. Chan, "Decision making for autonomous driving via augmented adversarial inverse reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 1036–1042.
- [29] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 1877–1901.
- [30] H. Du et al., "Chat with ChatGPT on intelligent vehicles: An IEEE TIV perspective," *IEEE Trans. Intell. Veh.*, vol. 8, no. 3, pp. 2020–2026, Mar. 2023.
- [31] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I the method of paired comparisons," *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952.
- [32] W. Xu, W. Zhaoyao, H. Zhao, and H. Zha, "A vehicle model for micro-traffic simulation in dynamic urban scenarios," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2011, pp. 2267–2274.
- [33] Z. Huang, H. Liu, J. Wu, and C. Lv, "Conditional predictive behavior planning with inverse reinforcement learning for human-like autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 7, pp. 7244–7258, Jul. 2023.
- [34] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, "Optimal trajectory generation for dynamic street scenarios in a frenet frame," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2010, pp. 987–993.
- [35] L. Chen et al., "Decision transformer: Reinforcement learning via sequence modeling," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 15084–15097.
- [36] M. Janner, Q. Li, and S. Levine, "Offline reinforcement learning as one big sequence modeling problem," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 1273–1286.
- [37] A. Radford et al., "Improving language understanding by generative pre-training," 2018.
- [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [39] Y. Bai et al., "Training a helpful and harmless assistant with reinforcement learning from human feedback," 2022, *arXiv:2204.05862*.
- [40] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2015, *arXiv:1506.02438*.
- [41] L. Engstrom et al., "Implementation matters in deep policy gradients: A case study on PPO and TRPO," 2020, *arXiv:2005.12729*.
- [42] L. Claussmann, M. Revilloud, D. Gruyer, and S. Glaser, "A review of motion planning for highway autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 5, pp. 1826–1848, May 2020.
- [43] E. Leurent, "An environment for autonomous driving decision-making," 2018. [Online]. Available: <https://github.com/eleurent/highway-env>
- [44] R. Krajewski, J. Bock, L. Kloecker, and L. Eckstein, "The highD dataset: A drone dataset of naturalistic vehicle trajectories on german highways for validation of highly automated driving systems," in *Proc. 21st Int. Conf. Intell. Transp. Syst.*, 2018, pp. 2118–2125.
- [45] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101*.
- [46] S. Teng et al., "Motion planning for autonomous driving: The state of the art and future perspectives," *IEEE Trans. Intell. Veh.*, vol. 8, no. 6, pp. 3692–3711, Jun. 2023.
- [47] R. Jena, C. Liu, and K. Sycara, "Augmenting gail with BC for sample efficient imitation learning," in *Proc. Conf. Robot Learn.*, 2021, pp. 80–90.



**Hejian Zhuang** received the B.S. degree in automotive engineering in 2022, from Tongji University, Shanghai, China, where he is currently working toward the M.S. degree. His research interests include intelligent vehicles, driving behavior modeling and driving decision-making.





**Hongqing Chu** received the B.S. degree in vehicle engineering from the Shandong University of Technology, Zibo, China, in 2012, and the Ph.D. degree in control theory and control engineering from Jilin University, Changchun, China, in 2017. He is currently an Associate Professor with the School of Automotive Studies, Tongji University, Shanghai, China. His research interests include optimal control of intelligent vehicle, advanced motion control of intelligent chassis, and advanced driver-assistance system.



**Bingzhao Gao** (Member, IEEE) received the B.S. and M.S. degrees in vehicle engineering from Jilin University, Changchun, China, in 1998 and 2002, respectively, and the Ph.D. degree in mechanical engineering from Yokohama National University, Yokohama, Japan, and in control engineering from Jilin University, in 2009. Since 2013, he has been a professor at Jilin University. He is currently a Professor with the College of Automotive Studies, Tongji University, Shanghai, China. His current research interests include vehicle powertrain control and vehicle stability control.



**Yafei Wang** (Member, IEEE) received the B.S. degree from Jilin University, Changchun, China, the M.S. degree from Shanghai Jiao Tong University, Shanghai, China, and the Ph.D. degree from The University of Tokyo, Tokyo, Japan. From 2013 to 2016, he was a Postdoctoral Researcher with The University of Tokyo. He is currently an Associate Professor with the School of Mechanical Engineering, Shanghai Jiao Tong University. His research interests include theory and application for automated driving vehicles.



**Hong Chen** (Fellow, IEEE) received the B.S. and M.S. degrees in process control from Zhejiang University, Hangzhou, China, in 1983 and 1986, respectively, and the Ph.D. degree in system dynamics and control engineering from the University of Stuttgart, Stuttgart, Germany, in 1997. In 1986, she joined the Jilin University of Technology, Changchun, China. From 1993 to 1997, she was a Wissenschaftlicher Mitarbeiter with the Institut fuer Systemdynamik und Regelungstechnik, University of Stuttgart. Since 1999, she has been a Professor with Jilin University, Changchun, and hereafter a Tang Aoqing Professor. From 2015 to 2019, she was the Director of the State Key Laboratory of Automotive Simulation and Control, Jilin University. She is currently a Distinguished Professor with Tongji University, Shanghai, China. Her research interests include model predictive control, nonlinear control, and applications in mechatronic systems focusing on automotive systems.